

Leveraging Large Language Models for Autonomous Cyber Defense: Insights from CAGE-2 Simulations

Hamoun Mohammadi¹, Defence and Security Institute, The University of Adelaide, Adelaide, SA, 5005, Australia

Jonathan J. Davis² and Mitchell Kiely², Defence Science and Technology Group, Edinburgh, SA, 5111, Australia

The ability of large language models (LLMs) to interpret and generate human-like text, along with their inbuilt knowledge and reasoning skills, has opened the door for many new applications. We develop a novel LLM agent as an autonomous cyber defender and benchmark it using the CAGE-2 public challenge, which evaluates submitted agents in a simulated network. The agent uses a pretrained LLM to interpret a text description of the challenge and choose the best defensive action given a network observation. Its performance is on par with the most successful reinforcement learning agent submitted to CAGE-2 with the advantages of explainability and avoiding training episodes, but with the cost of requiring LLM prompt engineering. Finally, we test the LLM agent experimentally to explore how LLM temperature and prompt complexity affect performance. We show that higher complexity prompts significantly improve performance, while LLM temperature has only a minor effect.

Government services online are exposed to cyber threats, including service disruption and information leakage.^{1,2} Defenses to these cyber threats are human-centric which, given the highly complex and interconnected digital infrastructure, results in slow response times and the risk of human error.

Autonomous cyber defense systems, driven by AI, offer a potential solution of near real-time threat detection and response with minimal human oversight. However, developing such systems poses difficulties, including ensuring accuracy and adapting to novel threats. The core goal is to create an autonomous cyber defense that scales and adapts in real-world environments with high reliability and trustworthiness.

Our approach for developing an autonomous cyber defense agent uses a pretrained large language model

(LLM) as a decision maker, leveraging the LLM's inbuilt knowledge and instruction-following abilities. We investigate whether the LLM can decide on a suitable sequence of actions to defend a simulated network from attack.

LLMs have already been applied to cybersecurity, including the study of an LLM agent to perform offensive operations in a simulation.³ However, there is a lack of research testing the suitability of LLM agents for defensive operations where the agent needs to balance the requirements of both stopping the red attacker and maintaining network functionality.

We test an LLM agent in the Cyber Autonomy Gym for Experimentation-2 (CAGE-2)⁴ public challenge. The challenge simulates a small enterprise network within the Cyber Operations Research Gym (CybORG⁵) and also provides automated red agent attackers for testing submissions of autonomous defenders. The simulation is a safe environment for testing and maturing autonomous agents and establishing any human-machine interactions. While CAGE-2 was designed for experimentation with reinforcement learning (RL)-based agents for cyber defense, CAGE-2 is also amenable

1541-1672 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.

Digital Object Identifier 10.1109/MIS.2025.3568209

Date of publication 8 May 2025; date of current version

11 August 2025.

to experimentation with other agent types, including LLM-based agents. This allows us to compare the performance of our LLM agent to previous RL submissions in CAGE-2.

The main contributions of the article are:

- › The development of an LLM agent for autonomous cyber defense in a simulated enterprise network environment. The agent's performance is compared to RL agents, and its advantages and limitations for network defense are discussed.
- › Experiments that test the effectiveness of prompt complexity and LLM temperature on LLM agent performance.

The rest of the article is organized as follows. The “Background” section discusses related work and information about the CAGE-2 public challenge. The “LLM Agent Architecture” section describes how our agent was developed, and then the “Agent Evaluation” section describes the experiments to measure its performance. The “Results” section details the experiment outcomes, which are then described in the “Discussion” section. The article’s “Conclusion” section outlines limitations and future work.

BACKGROUND

Related Works

LLMs can perform multiple tasks, including human-like text generation, text summarization, conversational question/answering, and computer code generation.⁶ LLMs have also shown promise in their ability to reason, plan, and self-reflect.⁷ These qualities have led to a surge in research to determine the effectiveness of LLMs within autonomous agents, including applications for robotics⁸ and AI task solvers.⁹

In the field of cybersecurity, LLM source code generation and interpretation skills have recently been used to build innovative applications. These include a penetration testing tool to generate one-day exploits from text descriptions of vulnerabilities,¹⁰ and a source code patching tool to scan for vulnerabilities and automatically apply patches.¹¹

Our approach is closely related to the LLM agent developed by Rigaki et al. for offensive cyber operations in simulation.³ Their agent runs in both their NetSecGame and CyberBattleSim¹² simulations with the task of choosing actions as a cyber attacker. It uses the GPT-4 LLM to choose the next action after providing the LLM a specially crafted prompt consisting of game instructions, memory, examples, and the current status. Similarly, we develop an LLM agent to choose actions within a simulation environment. However,

our approach differs first, because we take the side of blue, and second, because we experimentally analyze the effect of LLM temperature and prompt complexity on agent performance.

To enable the development of autonomous cyber agents, cyber simulation environments, such as Cyborg and CyberBattleSim, provide feedback to agents in the form of a reward. The rewards enable the training of RL agents via an OpenAI gym interface.¹³ Our autonomous LLM agent does not require training on episodic data per RL agents. However, reward signals serve as scoring mechanisms for comparison between LLM and RL agents.

When LLMs, such as GPT-3¹⁴ and Llama,¹⁵ were released, they demonstrated to the public significant advances in interpreting and generating human-like language and answering general knowledge questions. The models were trained on extensive language datasets curated from human contributions on the web, including publicly available cybersecurity information. We rely on the LLM to have some inbuilt cybersecurity knowledge so it can interpret the information provided to the LLM in each prompt about the current state of the network.

Despite the advances made with LLMs, they may still provide false or unhelpful answers to user queries. To overcome such issues, prompt engineering aims to guide LLM behavior to make responses more relevant, accurate, and consistent. Prompt engineering does not require modification to the LLM weights, but simply asks questions in a particular way and provides relevant context, e.g., the chain-of-thought prompt “Let’s think step by step” significantly increases performance on math benchmarks, such as GSM8K.⁷ Our agent uses ReAct-style prompting in a two-step “reason” and “act” process, along with few-shot learning to improve LLM responses.

CAGE-2

To evaluate an LLM agent in a dynamic environment, we used CAGE-2⁴ as it is considered¹⁶ a standard benchmark to compare methods for autonomous cyber defense. The CAGE-2 challenge requires users to create an autonomous blue agent tasked with the defense of a compact enterprise network simulated using Cyborg,⁵ as shown in Figure 1.

The challenge begins with the network in a post-initial exploit state in which the red agent has root access on a user host within Subnet 1. In one run of a game, known as a trial, a blue agent must defend the network against the red agent for a variable number of time steps.

The goal of the red agent is to move laterally within the enterprise network and gain access to

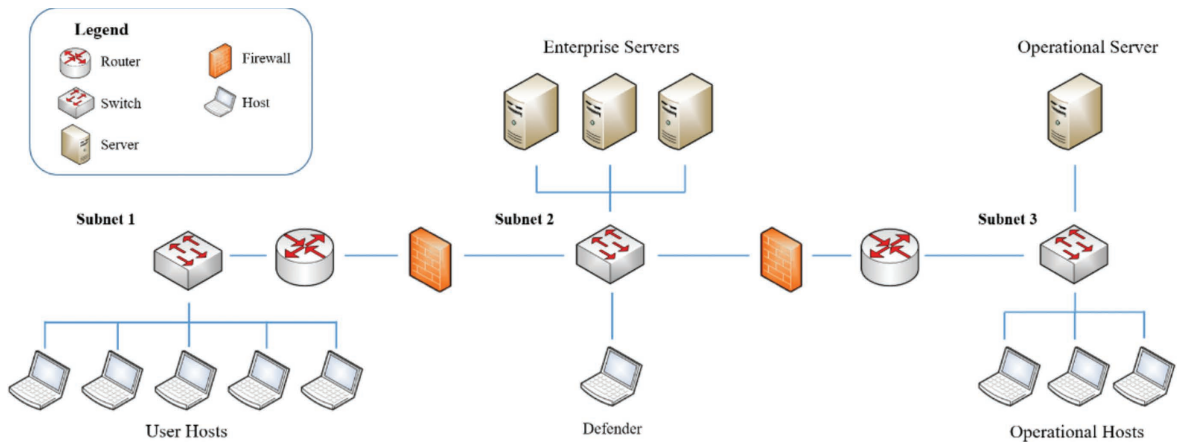


FIGURE 1. Enterprise network for CAGE-2 challenge.

the operational server. Three rules-based red agents are provided: *b-line*, *meander*, and *sleep*. This article focuses on defending the network against the *b-line* agent. The *b-line* agent executes actions in the least number of steps to reach and disrupt the operational server, thanks to prior network knowledge. The *meander* agent instead operates without any prior network knowledge, incrementally gaining privileged access to all hosts in a subnet before moving to the next subnet. The *sleep* agent serves as the benchmark and takes no offensive measures, allowing us to discern the blue agent's ability to recognize a nonthreatening situation. The blue agent's goal is to minimize the red agent's presence while sustaining network service availability.

The blue agent's action space is a set of 145 discrete actions. During each time step only one action can be selected by the blue agent. Apart from the sleep action, blue actions can be segregated into three distinct classes: reconnaissance, deception, and restoration. Reconnaissance actions enhance the blue agent's understanding of the current state of the network. Deception actions aim to slow down the red agent's movement by inserting decoys into the specified host. Finally, the restoration actions revert the host or server to a red agent-free state. Determining which action is the most effective to take at any given time step is entirely up to the autonomous agent.

LLM AGENT ARCHITECTURE

The LLM agent was designed to defend a network within the CybORG environment by preventing a red agent from reaching its target, the operational server. It consists of an LLM along with a small library of helper classes, methods, and prompts. The blue agent's library was designed to query the LLM in two separate

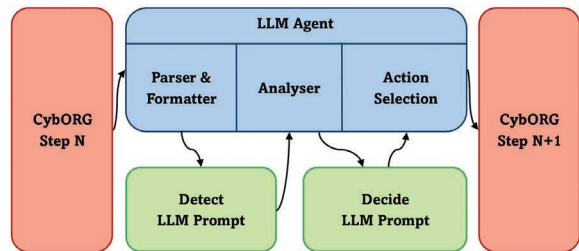


FIGURE 2. LLM agent architecture, with green boxes representing the calls made to the LLM.

tasks, each using a different prompt. The first task was to detect the current state of red agent threats, and the second task was to decide the best blue agent course of action. This multistep approach is similar to the "ReAct" prompting method.¹⁷ The first prompt to the LLM returns potential threats in the environment, which are then included in the second prompt as context to help the LLM decide the next cyber defense action.

The architecture of the blue agent is shown in Figure 2, with the purpose and operation of each module described in the following sections.

LLM Agent

At the start of a trial, CybORG provides the blue agent with an initial observation that details the network topology, including endpoint information, such as active processes, their privilege level, and open ports. The observation is sent to the agent's parser for processing.

Parser and Formatter

This module converts a CybORG observation into text format to enable interpretation using the LLM's innate

text-processing capabilities. The parser first extracts useful information from the current observation's data structure. Then it retrieves from agent memory the previous observation and agent action. The formatter integrates this information into a standardized string and provides it to the next module.

Detect LLM Prompt

The purpose of the blue agent's first LLM query is to detect what occurred in the most recent CybORG step, in particular red agent activity. To enable detection, we provide necessary information to the LLM in the form of a prompt which consists of three parts, as per the example in Appendix A of the supplementary information (available at <https://doi.org/10.1109/MIS.2025.3568209>, provided by the authors).

First, there is a worded explanation of the problem and the task to perform. Only enough information regarding the CybORG environment is provided for the LLM to do its intended tasks, resulting in the "game rules prompt." Second, there is a comprehensive static "examples prompt," utilizing few-shot and chain-of-thought strategies, with example CybORG observations presented in the format produced by the parser module. The examples demonstrate how to solve the problem step by step, with explicit statements regarding the decision logic so the LLM will base its own thinking on the examples. Third, the "observation prompt" lists the current and previous blue observations. These observations include hosts running in the network and their processes and connections.

We combine all three parts into a single prompt and send it to the LLM. The LLM uses this information to infer what action was performed in the CybORG environment during the last time step. Since only the red and blue agents perform actions, this enables the detection of current red agent threats. The LLM then returns the action as a JSON object showing the name of the host and the action performed, e.g. {"active": "Yes", "Enterprise0": "exploit"}.

Analyzer

The analyzer module receives the LLM answer stating the most recent action and updates the agent's internal state vector of network endpoints accordingly. The state vector specifies the current security status for each host as *safe*, *exploit*, *escalate*, or *decoy*. *Safe* means no detected red agent activity for the host, *exploit* means the red agent has gained user-level access to the host via an exploit, while *escalate* means the red agent has gained admin or root access. *Decoy* means the blue agent has added a decoy service to the host

with no detected red agent activity. When an endpoint is detected as exploited, the analyzer module checks the startup privilege level of the exploited process. If it is a root process, then the system is fully compromised and the state vector for that host is updated from *exploit* to *escalate*.

Decide LLM Prompt

The updated state vector of network endpoints plus the inferred last red agent action is then written into a new prompt for a second LLM query. An example of this "decide prompt" is shown in Appendix B of the supplementary information (available at <https://doi.org/10.1109/MIS.2025.3568209>, provided by the authors). The query asks the LLM to decide which blue agent action to perform given the information in the rest of the prompt.

Action Selection

The LLM's answer consists of the chosen action to be performed by the blue agent. The action is in string format, so this module converts it to a CybORG action to perform in the next time step. After both red and blue actions are taken, the agent reads the next CybORG observation and the process repeats.

AGENT EVALUATION

We evaluated the LLM agent in online and offline scenarios. In the online assessment, the blue LLM agent performed defensive actions in the CAGE-2 environment to prevent the b-line red agent from furthering its foothold in the network. The b-line agent was selected because it represented the worst-case scenario, where an attacker possesses knowledge of the target network, enabling rapid navigation through the subnets to inflict maximum damage in the fewest possible actions. Therefore, this evaluation measured the LLM agent's ability to execute a series of actions that minimized this damage. On the other hand, the offline evaluation, which utilized extracted simulation data, was conducted to test the LLM's ability to select the optimal action based on a given observation. This evaluation was performed across a range of LLM temperatures and varying levels of prompt complexities. For all experiments, OpenAI's GPT-4 was prompted as the LLM, since it produced more accurate and consistent outputs compared to GPT-3.5, Falcon-7B, and smaller locally hosted models.

Online Evaluation

Measuring the effectiveness of the LLM agent was evaluated with a reward-based method. The reward

system closely aligns with the blue agent's goals by: 1) penalizing the blue agent whenever the red agent has root access to a host/server, 2) penalizing the blue agent for executing the costly restorative actions, and 3) penalizing the blue agent each time the red agent disrupts the operational server. This is represented in the following equation:

$$\text{Reward}_{\text{Blue}} = -0.1(\text{host}_{\text{Red}} - 1) - \text{server}_{\text{Red}} - \text{Blue}_{\text{Restore}} - 10(\text{Red}_{\text{impact}}).$$

host_{Red} and $\text{server}_{\text{Red}}$ represent the number of servers and hosts, respectively, that the red agent has root access to. The -1 after host_{Red} accounts for the initial foothold which the red agent commences with, which cannot be removed. $\text{Blue}_{\text{Restore}}$ and $\text{Red}_{\text{impact}}$ are binary variables that equate to 1 if the representation actions were taken in a given time step or 0 otherwise. Accordingly, the maximum reward an agent can obtain is zero. The blue LLM agent was evaluated against the red b-line agent across 30 trials, where each trial consisted of 30 time steps. The number of trials was limited to 30 due to the LLMs slow inference rate, resulting in slow action selection compared to RL agents. The number of time steps per trial was also chosen as 30 due to b-line's deterministic behavior, which follows a 15-step kill-chain. Thus, increasing the number of time steps per trial is unlikely to offer any new insights.

We took the reward score from each trial to calculate the average reward and the standard deviation, from which we derived a 95% confidence interval. We then compared the calculated reward confidence interval for the LLM agent, with the results from the victor of the CAGE-2 challenge, an RL-based agent trained in the same scenario.

Offline Evaluation

We conducted an offline evaluation of the LLM by running tests using observations from the online evaluation. We selected 50 unique observations from the logs of the online evaluation to test the LLM's reasoning abilities to select the optimal action for a given observation.

We evaluated the LLM's performance using three different classes of prompts with increasing detail and complexity at four distinct model temperatures. Subject matter experts determined what the optimal action was for each of the 50 observations. We then passed the prompt and the observation to the LLM, recorded the action that was generated, and compared this output to our determined optimal action.

We conducted this test for all 50 observations, across all prompt complexities and model temperatures. We scored as a failure any response that did not exactly match the desired output as that response would have led to a nonoptimal action within a Cyborg game.

To best describe and quantify prompt complexity, we created three categories: low-, medium-, and high-complexity prompts. Each represented the varying level of detail provided in the prompt regarding the simulation and task to perform.

Low-complexity prompting is shown as the combination of the "game rules prompt (low complexity)" and "observations prompt" in Appendix A of the supplementary information (available at <https://doi.org/10.1109/MIS.2025.3568209>, provided by the authors). We defined a low-complexity prompt as one written with minimal information, containing vague and incomplete explanations, and with no example cases. We wrote the explanation in an intentionally vague manner, as would a novice who might expect the LLM to understand the nuances of the task by itself.

For the medium-complexity prompt, we included a much more detailed initial explanation of the task and rules, as well as a noncomprehensive example set of six examples, containing only the most frequently appearing inputs. We wrote the examples followed by the correct response to them and omitted the intermediate steps and reasoning, only telling the model to work step by step to reach the correct answer, a technique known as chain-of-thought. Although we provided the LLM with much more context, the limited scope of these examples and the lack of scrutiny in their explanations made this class of prompting significantly less detailed than the following style.

High-complexity prompts have the same detailed explanation as the medium-complexity prompt, paired with an extended example set, intended to be much more comprehensive and encapsulate the many differing potential observations. As shown in Appendix A of the supplementary information (available at <https://doi.org/10.1109/MIS.2025.3568209>, provided by the authors), we provided the LLM with highly detailed example explanations that contained step-by-step reasoning. Our high-complexity prompt was developed in the final iteration of the LLM agent and is the most specialized for the task. This prompt utilized chain-of-thought and few-shot learning prompting strategies.

LLM temperature controls the randomness of the LLM output. A temperature closer to zero results in more deterministic output, in that the highest probability next token is always selected. As the temperature

increases, more weight is given to other possible tokens, leading to more diverse outputs.

We tested the LLM across all 50 examples in 12 different experiment configurations. Each experiment used a combination of prompt complexity (low, medium, or high) and LLM temperature (0.25, 0.6, 0.9, or 1.1) to observe the effects of these factors on the correctness of the LLM responses.

RESULTS

In the online evaluation, we collected rewards from trials played with the LLM agent to measure the agent's performance. Table 1 shows the 95% confidence interval for the reward for the LLM agent alongside that of the top-performing RL agent. Since the confidence intervals overlap, there is no statistical difference in their performance. The interval between lower and upper bound is 0.66 for the LLM agent and 0.21 for the RL agent. The smaller interval for the RL agent is due to performing many more trials for that agent.

In the offline evaluation, we assessed the accuracy of the LLM's action selection across various prompt complexities and LLM temperatures. The results in Figure 3 show that LLM accuracy in producing an exact output is very sensitive to prompt complexity. The

TABLE 1. Comparison of top RL agent and LLM agent reward performance with 95% confidence intervals against the b-line agent for 30 time steps.

Agent Type	RL Agent	LLM Agent
Mean reward	-3.47	-3.51
Lower bound	-3.58	-3.84
Upper bound	-3.37	-3.18
Number of trials	1000	30

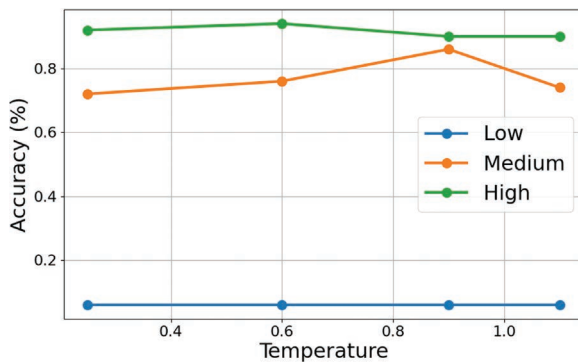


FIGURE 3. Visual representation of accuracy against LLM temperature and prompt complexity.

high-complexity prompt produces the highest accuracy. However, varying the LLM temperature results in only minor changes to LLM accuracy.

DISCUSSION

The online evaluation demonstrated that LLM agents can achieve results comparable to RL agents in autonomous cyber defense. This is shown in Table 1, where only a 0.04 difference in the average reward exists between the two agents. The significant overlap in their 95% confidence intervals suggests that the LLM agent matched the performance of the most capable RL agent against the b-line red agent.

The comparison between the LLM and RL agents is fair as both implemented their algorithms specifically to counter the b-line red agent, and they were both developed within the same allocated timeframe. From the RL perspective, the blue agent was specifically trained against only the b-line red agent. On the other hand, from the LLM perspective, the LLM prompt was designed with the b-line's strategy in mind, including telling the LLM to prioritize defense of critical hosts that we expect the b-line agent will target.

We found the LLM agent did not generalize particularly well to a different attacker. We conducted a simple test with an unchanged blue LLM agent defending against the meander red agent for an equal number of time steps. The rewards obtained were not as competitive as the top-performing RL agent, which was trained specifically against the meander agent. This outcome is attributed to the current LLM prompt being designed solely to counter the b-line red agent. Additionally, since the meander agent explores the network in a different manner, many environment observations are out-of-distribution states compared to the example observations listed in our LLM prompt.

One advantage of using LLM agents over RL agents is their explainability. While the LLM model itself is a black box, it can provide a step-by-step explanation of how it generated an answer in response to a query. This allows a user to comprehend why the LLM selected the action it executed, whereas RL models provide no such insights.

Another advantage is the flexibility of prompt engineering. LLM agents offer a high potential for modular expansions to prompts, allowing changes to processing logic and swift and streamlined redeployment. This is in contrast to the time and computing power required to train RL models, especially for emulation of real-world environments.

A disadvantage of using LLM agents is their slower inference time when selecting actions. Additionally, the model complexity, in terms of number of model

parameters, is significantly higher for pretrained LLMs compared to RL models. Finally, the prompt engineer requires a comprehensive understanding of the task to effectively instruct the LLMs.

The offline evaluation measures the effect of prompt complexity and LLM temperature on the LLM's ability to select the optimal action for a given observation. The offline dataset consists of 50 observations generated from b-line agent runs, with each observation being a different state configuration. As such, these observations represent a diverse set for LLM agent evaluation. The offline evaluation used the same 50 observations when testing different LLM temperature and prompt complexity pairings.

We found that prompt complexity was the most critical factor in selecting the optimal action. High-complexity prompts consistently outperformed both medium- and low-complexity prompts across all temperature ranges. Higher-complexity prompts were more detailed and included more examples. Fewer examples force the LLM to generalize its knowledge to out-of-distributions states, i.e., observations they have not seen before. The results suggest that LLM agents struggle to generalize from limited information.

For the low-complexity prompt, changes in the model temperature did not significantly affect the model's performance. However, modifying the LLM's temperature did impact the performance in the medium- and high-complexity prompts. The medium-complexity prompt performed best at a temperature of 0.9, which resulted in an accuracy increase of 14%. The high-complexity prompt performed best at a temperature of 0.6, with only a 4% variation between its highest and lowest performing temperatures. We surmise the very specific instructions in the high-complexity prompt are suited to a lower LLM temperature for predictable responses.

We used GPT-4 as the LLM in our agent architecture as it consistently performed better than other models, such as GPT-3.5 and smaller locally hosted models. The drawback of this powerful model is higher prompting token costs and longer response times associated with each prompt. In the offline and online evaluation, where time associated with decision making does not impact performance, powerful models are more advantageous. In a more realistic setting, where the time required to execute actions impacts the results, a more rigorous model selection process will be required.

When selecting different temperatures, we tested up to a value of 1.1, as higher temperatures led to inaccurate LLM responses with excessive hallucination.

Temperatures lower than 0.25 also produced significantly inferior results as they interpreted the examples too literally and failed to recognize patterns and adapt to differing circumstances.

CONCLUSION

We demonstrated the use of prompt engineering to successfully guide a pretrained LLM in the role of an autonomous blue agent in a cyber simulation challenge. The performance of the LLM agent matched that of RL agents for the task. Experiments also showed the benefit of writing detailed prompts. However, there are currently several limitations to our approach.

Real-World Applicability

Our experiments were performed in simulation. Whether LLM agents can succeed in real-world, unconstrained environments is unknown. Additionally, our experiments were performed against a single red agent. To defend a network against a range of attackers, the LLM agent prompts would need to be extended. While prompt engineering is easily extensible, it is unclear how well this approach would scale to counter numerous different attack vectors.

Computing Resources

LLMs require significant computing resources at inference time. They are more costly to run than smaller, single-task AI models, such as an RL model. Therefore, LLM agents must offer more than matching the performance of an RL model to justify the additional expense and hardware demands. Future work could quantify additional benefits, such as explainability or prompt flexibility, to justify the LLM agent approach.

LLM Maturity and Security

Since LLMs are a relatively recent technology, new models and versions are frequently released, each with different capabilities. Even the same model can produce responses that vary over time. Since results are not always repeatable, justifying the reliability of an LLM agent for mission-critical applications is difficult. Currently, closed pretrained LLMs such as GPT-4 are the most capable. These models are only available via API calls over the Internet, therefore risking information leakage or service unavailability due to connection issues. Government agencies may therefore require a more robust solution, such as an on-premise, local LLM to retain data ownership and for reproducible results.

ACKNOWLEDGMENTS

This article has supplementary downloadable material available at <https://doi.org/10.1109/MIS.2025.3568209>, provided by the authors.

REFERENCES

1. "APT40 advisory: PRC MSS tradecraft in action." ACSC. Accessed: Feb. 21, 2024. [Online]. Available: <https://www.cyber.gov.au/about-us/view-all-content/alerts-and-advisories/apt40-advisory-prc-mss-tradecraft-in-action>
2. "SVR cyber actors adapt tactics for initial cloud access." ACSC. Accessed: Feb. 21, 2024. [Online]. Available: <https://www.cyber.gov.au/about-us/view-all-content/alerts-and-advisories/svr-cyber-actors-adapt-tactics-initial-cloud-access/>
3. M. Rigaki, O. Lukáš, C. A. Catania, and S. Garcia, "Out of the cage: How stochastic parrots win in cyber security environments," 2023, *arXiv:2308.12086*.
4. M. Kiely, D. Bowman, M. Standen, and C. Moir, "On autonomous agents in a cyber defense environment," 2023, *arXiv:2309.07388*.
5. M. Standen, M. Lucas, D. Bowman, T. J. Richer, J. Kim, and D. Marriott, "CybORG: A gym for the development of autonomous cyber agents," 2021, *arXiv:2108.09118*.
6. A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
7. S. Yao et al., "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 11,809–11,822.
8. D. Driess et al., "PaLM-E: An embodied multimodal language model," 2023, *arXiv:2303.03378*.
9. S. Yongliang, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, "HuggingGPT: Solving AI tasks with chatgpt and its friends in hugging face," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2024, vol. 36, pp. 38,154–38,180.
10. R. Fang, R. Bindu, A. Gupta, and D. Kang, "LLM agents can autonomously exploit one-day vulnerabilities," 2024, *arXiv:2404.08144*.
11. P. Tempel and E. Tooley, "Found means fixed: Introducing code scanning autofix, powered by GitHub Copilot and CodeQL." *The GitHub Blog*, Mar. 20, 2024. [Online]. Available: <https://github.blog/2024-03-20-found-means-fixed-introducing-code-scanning-autofix-powered-by-github-copilot-and-codeql/>
12. C. Seifet et al. "Microsoft defender research team: CyberBattleSim." GitHub. Accessed: Apr. 5, 2024. [Online]. Available: <https://github.com/microsoft/cyberbattlesim>
13. G. Brockman et al., "OpenAI gym," 2016, *arXiv:1606.01540*.
14. T. B. Brown et al., "Language models are few-shot learners," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.
15. H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
16. K. Hammar, N. Dhir, and R. Stadler, "Optimal defender strategies for CAGE-2 using causal modeling and tree search," 2024, *arXiv:2407.11070*.
17. S. Yao et al., "React: Synergizing reasoning and acting in language models," 2022, *arXiv:2210.03629*.

HAMOUN MOHAMMADI is a graduate software engineer at Fivecast at Adelaide, SA, 5000, Australia and the University of Adelaide, Adelaide, SA, 5005, Australia. His research interests include cybersecurity, open source intelligence, and artificial intelligence. Mohammadi received his B.S. degree in mathematical and computer sciences from the University of Adelaide. Contact him at hamoon.mohammadi@gmail.com.

JONATHAN J. DAVIS is a cybersecurity researcher at Defence Science and Technology Group in Edinburgh, SA, 5111, Australia. His research interests include machine learning, cybersecurity, and autonomy. Davis received his Ph.D. degree in cybersecurity from the Queensland University of Technology. Contact him at jonathan.davis@dst.defence.gov.au.

MITCHELL KIELY is a cybersecurity researcher at Defence Science and Technology Group in Edinburgh, SA, 5111, Australia. His research interests include cybersecurity and agent artificial intelligence. Kiely received his Honors degree in physics from the University of New South Wales. Contact him at mitchell.kiely@dst.defence.gov.au.